# Ders 3 Genişletilmiş Notu: Insertion Sort, Asal Sayı Bulma, İkili Arama ve Algoritma Analizine Giriş

## Ders 3 Genişletilmiş Notu: Insertion Sort, Asal Sayı Bulma, İkili Arama ve Algoritma Analizine Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Önceki Dersin Hatırlatması
- 2. Insertion Sort (Araya Ekleyerek Sıralama)
- 3. Asal Sayı Problemi ve Adım Adım İyileştirme
- 4. Eratosthenes Kalburu (Sieve of Eratosthenes)
- 5. Problem Çözerken İzlenecek Yol: Önce Doğru, Sonra Hızlı, Sonra Az Bellekli
- 6. Arama Algoritmaları: Doğrusal Arama, İkili Arama ve Hash
- 7. İkili Aramanın Akış Diyagramı ve Adım Adım İzlenmesi
- 8. Algoritma Analizi: Temel İşlemler ve Big O
- 9. En İyi, Ortalama ve En Kötü Durum
- 10. Zaman Karmaşıklığını Belirleyen Etkenler
- 11. Büyüme Dereceleri
- 12. Asimptotik Notasyon: Big O, Big Omega, Big Theta
- 13. Sözde Kod ve En Büyük Elemanı Bulma
- 14. Teorik Analiz ve Deneysel Ölçüm


## 1. Önceki Dersin Hatırlatması

Hoca derse, geçen hafta yapılanları kısaca hatırlatarak başladı. Geçen hafta temel sıralama algoritmalarından söz edilmişti. Bir algoritma tasarlanırken akış diyagramında hangi sembollerin kullanılması gerektiği, diyagramın nasıl çizilmesi gerektiği üzerinde durulmuştu. Bir algoritma tasarlarken ya da iki algoritmayı karşılaştırırken nelere dikkat edileceği de kabaca konuşulmuştu. Geçen hafta selection sort ve bubble sort görülmüş, ders insertion sort'ta kalmıştı. Bu ders oradan devam etti.

Hocanın iPad'i ekrana aktarılamadığı için ders, önceden hazırlanmış slaytlar üzerinden anlatıldı. Hoca bazı yerlerde kalemle yazmak için boş sayfa bırakmıştı, ama bu sayfalar kullanılamadı. Bu nedenle bazı kod ve akış diyagramı ayrıntıları yalnızca sözlü anlatımdan kurulmuştur.

## 2. Insertion Sort (Araya Ekleyerek Sıralama)

### 2.1 Fikir: Elimizdeki İskambil Kâğıtlarını Dizmek

Insertion sort, selection sort ve bubble sort'tan sonra görülen üçüncü temel sıralama algoritmasıdır. Hocanın benzetmesi iskambil oyunudur: Kâğıtlar elinize rastgele gelir. Her yeni kâğıdı alır, elinizdeki sıralı kâğıtlar arasında olması gereken yere yerleştirirsiniz. Insertion sort da diziyi bu fikirle sıralar. Dizinin sol tarafı her an sıralıdır. Sıradaki eleman, bu sıralı kısmın içinde doğru yerine yerleştirilir.

Doğal olarak ikinci elemandan başlanır. Tek elemanlı bir parça zaten sıralıdır.

### 2.2 Derste İzlenen Örnek

Derste kullanılan dizi şudur: 5, 1, 4, 3, 2, 7, 6.

- **1'in yerleştirilmesi:** 1 alınır. 1, 5'ten küçük olduğu için 5 bir sağa kayar, 1 başa gelir. Dizi: 1, 5, 4, 3, 2, 7, 6.
- **4'ün yerleştirilmesi:** 4 alınır. 4, 5'ten küçük olduğu için 5 sağa kayar. 4, 1'den küçük olmadığı için durulur. 4, 1 ile 5 arasına yerleşir. Dizi: 1, 4, 5, 3, 2, 7, 6.
- **3'ün yerleştirilmesi:** 3 alınır. 3, 5'ten küçük olduğu için 5 sağa kayar. 3, 4'ten de küçük olduğu için 4 de sağa kayar. Bu anda dizinin başı 1, 4, 4, 5 görünümündedir: 4 hem eski yerinde durur hem de bir sağa kopyalanmıştır. 3, 1'den küçük olmadığı için döngü biter. 3, açılan yere (eski 4'ün üzerine) yazılır. Dizi: 1, 3, 4, 5, 2, 7, 6.
- Aynı işlem 2, 7 ve 6 için tekrarlanır.

Hoca 2, 7 ve 6 adımlarını yalnızca "aynı işlemle devam ediyoruz" diyerek geçti. Aşağıdaki tablo bütün turları gösterir (ara adım eklendi):

| Tur ($i$) | Alınan eleman (`tmp`) | Sağa kayan elemanlar | Tur sonunda dizi |
|---|---|---|---|
| 1 | 1 | 5 | 1, 5, 4, 3, 2, 7, 6 |
| 2 | 4 | 5 | 1, 4, 5, 3, 2, 7, 6 |
| 3 | 3 | 5, 4 | 1, 3, 4, 5, 2, 7, 6 |
| 4 | 2 | 5, 4, 3 | 1, 2, 3, 4, 5, 7, 6 |
| 5 | 7 | yok | 1, 2, 3, 4, 5, 7, 6 |
| 6 | 6 | 7 | 1, 2, 3, 4, 5, 6, 7 |

Bu örnekte toplam 8 kaydırma ve 13 karşılaştırma (`a[j] > tmp` sınaması) yapılır.

### 2.3 Akış Diyagramı ve Kod

Akış diyagramında önce dizinin boyutu, sonra dizinin kendisi okunur. Ardından iç içe iki döngü kurulur. Hoca burada genel bir gözlem yaptı: Karşılaştırma yaparak sıralayan algoritmaların mutlaka iki döngüye ihtiyacı vardır ve sıralama algoritmalarının büyük çoğunluğu karşılaştırma yaparak çalışır. Hoca, karşılaştırma dışında bir yaklaşımla sıralayan algoritmaların da ileride (bu sene ya da sonraki sene) görüleceğini söyledi.

Algoritmanın adımları:

- Dış döngü $i = 1$'den, yani dizi 0'dan başladığı için ikinci elemandan başlar ve son elemana kadar gider.
- Her turda o turun elemanı `tmp` değişkenine alınır.
- `j`, `i - 1` olarak ayarlanır. Böylece her seferinde hangi elemanla karşılaştırma yapılacağı bilinir.
- İç döngü, `j >= 0` olduğu ve `a[j] > tmp` olduğu sürece döner. Her adımda `a[j+1] = a[j]` yapılarak o anki eleman bir sağa kaydırılır, sonra `j` bir azaltılır.
- İç döngü iki durumda biter: ya dizinin başına varılmıştır ya da `a[j]` artık `tmp`'den büyük değildir.
- Döngüden çıkınca elimizdeki eleman açılan yere yazılır: `a[j+1] = tmp`.

Dersteki diziyle çalışan tam program:

```c
#include <stdio.h>

int main(void) {
    int a[] = {5, 1, 4, 3, 2, 7, 6};
    int n = sizeof(a) / sizeof(a[0]);
    int i, j, tmp;

    for (i = 1; i < n; i++) {
        tmp = a[i];
        j = i - 1;
```

```
        while (j >= 0 && a[j] > tmp) { /* önce sınır kontrolü */
            a[j + 1] = a[j];           /* sağa kaydır */
            j--;
        }
        a[j + 1] = tmp;                /* açılan yere yerleştir */
    }

    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Program `gcc -Wall` ile uyarısız derlenir ve `1 2 3 4 5 6 7` yazdırır.

**Sınır kontrolü önce yazılır** Hoca burada özellikle durdu. İç döngüde iki koşul vardır ve ilk kontrol sınır kontrolüdür (`j >= 0`). Tasarlanan algoritmalarda sınır kontrolleri genellikle ilk adımda yapılmalıdır. Kontrol ikinci sıraya bırakılırsa, `j` $-1$ olduğunda önce `a[j] > tmp` hesaplanmaya çalışılır, yani `a[-1]`'e erişilir. Size ait olmayan bir bellek alanına erişmek segmentation fault hatasına yol açabilir ve işletim sistemi programı sonlandırır.

> **Hocanın vurgusu:** Bir döngü koşulunda birden fazla kontrol varsa ve bunlardan biri sınır kontrolüyse, sınır kontrolünü ilk sıraya yazın.
>
> **Ek bilgi (derste anlatılmadı):** C'de && işleci kısa devre (short-circuit) değerlendirme yapar. Soldaki koşul yanlışsa sağdaki koşul hiç hesaplanmaz. Bu yüzden `j >= 0 && a[j] > tmp` yazıldığında `j` $-1$ olunca `a[-1]` okunmaz. Koşullar ters sırada yazılırsa okuma, sınır kontrolünden önce gerçekleşir.

**AND mi, OR mu?** İki koşulun aynı anda sağlanması gerekiyorsa AND kullanılır. Koşullardan birinin sağlanması döngüde kalmak için yetiyorsa OR kullanılır. Insertion sort'ta hem `j`'nin 0'a eşit ya da 0'dan büyük olması hem de `tmp`'nin `a[j]`'den küçük olması gerekir. Bu yüzden iki koşul AND ile bağlanır.

### 2.4 En İyi ve En Kötü Durum

Hoca, sıralı bir dizi verildiğinde ne olacağını adım adım gösterdi. Dizi 1, 2, 3, 4, 5, 6, 7 olsun. 2 alınır, 1'den küçük mü diye bakılır: değil. Hiçbir şey yapılmaz, iç döngüden çıkılır, dış döngüye dönülür. Sonra 3, 2'den küçük mü? Hayır. 4, 3'ten küçük mü? 5, 4'ten küçük mü? 6, 5'ten küçük mü? 7, 6'dan küçük mü? Her turda tek bir soru yeter. İç döngünün gövdesine hiç girilmez.

Böylece sıralı bir dizide, dizinin sıralı olduğu en az işlemle anlaşılır. Bu en iyi durumdur (best case) ve yaklaşık $n$ karşılaştırma gerektirir.

> **Ders sonrası not:** Hoca en iyi durumda "$n$ tane karşılaştırma" dedi. Tam sayı $n-1$'dir, çünkü dış döngü ikinci elemandan başlar. Derste sayılan sorular da 6 tanedir (2-1, 3-2, 4-3, 5-4, 6-5, 7-6), dizi ise 7 elemanlıdır. Big O açısından fark yoktur: en iyi durum $O(n)$'dir.

En kötü durumda dizi büyükten küçüğe sıralıdır. Bu durumda her eleman en başa kadar kaydırılır. İç içe iki döngü tam olarak çalışır ve $n^2$ mertebesinde karşılaştırma yapılır, yani karmaşıklık $O(n^2)$'dir. 7 elemanlı ters sıralı bir dizide karşılaştırma sayısı $\frac{7 \cdot 6}{2} = 21$'dir (ara adım eklendi).

### 2.5 Insertion Sort Neden Tercih Edilir?

Hocaya göre bu yaklaşımın iki güzel tarafı vardır:

- Yer değiştirme (swap) işlemi yapılmaz. Yalnızca kaydırma yapılır.
- Gerektiği kadar eleman kaydırılır. Eleman yerini bulduğu anda tur biter.

Bu nedenlerle insertion sort küçük boyutlu dizilerde çok kullanışlıdır ve selection sort ile bubble sort'tan daha çok tercih edilir. Hem işlem sayısı azdır hem de en iyi duruma yakın, yani neredeyse sıralı dizilerde işi çok daha az işlemle bitirir.

Hoca insertion sort hakkında soru sordu, soru gelmedi.

## 3. Asal Sayı Problemi ve Adım Adım İyileştirme

### 3.1 Problem ve Önemi

Üç temel sıralama algoritmasından sonra hoca önemli bir problemi ele aldı: Verilen bir sayının asal olup olmadığına nasıl karar verilir? Ya da $n$'e kadar olan asal sayılar nasıl bulunur?

Asal sayı (prime number), yalnızca 1'e ve kendisine tam bölünebilen, bunların dışındaki hiçbir sayıya tam bölünemeyen sayıdır.

Asal sayılar iki alanda önemlidir:

- Rastgele sayı üreteçlerinde asal sayılardan yararlanılır.
- Kriptolojide asal sayılardan yararlanılır. Şifrelemede büyük asal sayılar çok işe yarar.

Bu yüzden yalnızca küçük asal sayılarla değil büyükleriyle de ilgilenilir. Hoca örnek olarak 987.972 gibi büyük bir sayının asal olup olmadığını bulmanın önemli olduğunu söyledi.

> **Ders sonrası not:** 987.972 çift sayıdır, dolayısıyla asal değildir (2'ye bölünür). Hoca bu sayıyı yalnızca "büyük bir sayı" örneği olarak verdi, asal olduğunu söylemedi.

Bu problem birinci sınıftan beri ele alınır. Çözümün farklı verimlilik düzeyleri vardır. Hoca problemi adım adım iyileştirerek ele aldı.

### 3.2 Kaba Kuvvet Çözüm (Naive Solution, Brute Force)

Naive solution ya da brute force approach, bir probleme ilk akla gelen çözümü uygulama yaklaşımıdır. Sofistike olmayan, herkesin aklına gelebilecek çözümdür.

Bir sayının asal olup olmadığını kâğıt üzerinde araştırsaydınız, sayıyı sırayla küçük sayılara bölüp tam bölünüp bölünmediğine bakardınız. Örneğin 37 için 2'ye, 3'e, 4'e, 5'e bölmeyi denersiniz ve bunu 36'ya kadar sürdürürsünüz. 2'den 36'ya kadar hiçbir sayıya bölünmüyorsa 37 asaldır. Herhangi birine bölünüyorsa asal değildir.

### 3.3 Birinci İyileştirme: Yarıya Kadar Bakmak

36'ya kadar gitmeden bu iş çözülebilir mi? Evet. Hoca 36 sayısını örnek aldı. 36; 2'ye, 3'e, 4'e, 6'ya, 9'a bölünür. Bölenleri ikililer halinde yazınca şu çiftler çıkar: (2, 18), (3, 12), (4, 9). 2'ye bölünen bir sayı 18'e de bölünür, 3'e bölünen 12'ye de bölünür. Yani sayının yarısından büyük bölenleri ayrıca kontrol etmeye gerek yoktur. Karşılaştırmayı sayının kendisine kadar değil, yarısına kadar yapmak akıllıca görünür.

### 3.4 İkinci İyileştirme: Kareköke Kadar Bakmak

Bu fikir ilerletilince yarıya kadar gitmenin de gereksiz olduğu görülür. 36 için 18'e kadar bakmaya gerek yoktur. Bölen çiftleri incelendiğinde, sayının kareköküne kadar ilerlemek yeterlidir. 36'nın son bölen çifti (6, 6)'dır (ara adım eklendi). 37 için 2, 3, 4, 5 ve 6'yı kontrol etmek yeter. Hoca "7'yi de kontrol edelim" dedi. Bu sayılardan hiçbirine bölünmüyorsa 37 asaldır.

> **Ek bilgi (derste anlatılmadı):** Karekökün neden yettiği şöyle gösterilir. $x = a \cdot b$ ve $a \leq b$ ise $a \cdot a \leq a \cdot b = x$ olur, yani $a \leq \sqrt{x}$'tir. Bu yüzden $x$ bileşik bir sayıysa en az bir böleni $\sqrt{x}$'ten küçük ya da ona eşittir. $\sqrt{x}$'e kadar bölen bulunamazsa sayı asaldır.

### 3.5 Üçüncü İyileştirme: 2'yi Ayrı Kontrol Edip Yalnız Tek Sayılara Bölmek

Son iyileştirmede önce sayının 2'ye bölünüp bölünmediğine bakılır. Sayı 2'ye bölünüyorsa asal olma ihtimali ortadan kalkar. Bölünmüyorsa, bölme denemeleri 3'ten başlar ve her seferinde

bir yerine iki artırılır. Çünkü 2'ye bölünmeyen bir sayı 4'e de, 6'ya da bölünmez. Böylece yalnızca tek sayılar denenir: 3, 5, 7, 9, ... Bu, kontrol sayısını önemli ölçüde azaltır.

Hocanın anlattığı akış:

- $x$ alınır. $i = 3$ yapılır. Sınır değer olarak $\sqrt{x}$ belirlenir.
- $x$ mod 2 sıfırdan farklı mı diye bakılır. Sıfırsa sayı asal değildir.
- Sınır aşılmadığı ve $x$ mod $i$ sıfırdan farklı olduğu sürece $i$ ikişer artırılır.
- Döngüden çıkınca, sınırın aşılıp aşılmadığına bakılır. Sınır aşılmışsa hiçbir bölen bulunmamıştır, sayı asaldır ("prime number"). Aşılmamışsa döngü bir bölen bulduğu için bitmiştir, sayı asal değildir ("not a prime number").

**101 örneği:** $\sqrt{101} \approx 10{,}05$ olduğu için 10'a kadar gidilir. 2'ye ilk adımda bakılmıştır, 101 2'ye bölünmez. Sonra 3, 5, 7 ve 9 denenir. Hiçbirine bölünmez. 11 sınırı aştığı için işlem biter ve 101 asal bulunur.

> **Ders sonrası not:** Hoca döngü sonunda "$i$ eşit $x$ ise prime number" diye bir kontrol anlattı, ardından "sınır artı bir" ifadesini kullandı. $i$ hiçbir zaman $x$'e ulaşmaz, çünkü döngü sınırda durur. Doğru kontrol, döngüden sınırı aşarak çıkılıp çıkılmadığıdır (`i > sinir`, sınır $\lfloor\sqrt{x}\rfloor$ alındığında). Döngü koşulunda sınır dahil edilmelidir (`i <= sinir`). Koşul `i < sinir` yazılıp sonuç `i >= sinir` ile yorumlanırsa 9, 15, 25, 35, 49 gibi sayılar yanlışlıkla asal bulunur (3 ile 59 arasındaki tek sayılar üzerinde denenerek bulundu).
>
> **Ders sonrası not:** `x % 2 == 0` kontrolü 2'yi de "asal değil" sayar, oysa 2 asaldır. 1 de asal değildir. Bu iki durum ayrıca ele alınmalıdır.

Aşağıdaki program dersteki akışı uygular. 2 ve 2'den küçük sayılar için iki satır ders sonrası eklenmiştir:

```c
#include <stdio.h>
#include <math.h>

/* 1: asal, 0: asal değil */
int asal_mi(int x) {
    int i, sinir;
    if (x < 2)
        return 0;
    if (x == 2)                 /* ders sonrası eklendi */
        return 1;
    if (x % 2 == 0)
        return 0;
    sinir = (int)sqrt(x);
    i = 3;
    while (i <= sinir && x % i != 0)
        i = i + 2;
    return i > sinir;           /* sınır aşıldıysa bölen bulunmadı */
}

int main(void) {
    int sayilar[] = {37, 101, 987972};
    int k;
    for (k = 0; k < 3; k++)
        printf("%d: %s\n", sayilar[k],
               asal_mi(sayilar[k]) ? "prime number" : "not a prime number");
    return 0;
}
```

Program `gcc -Wall asal.c -lm` ile uyarısız derlenir. Çıktısı: 37 ve 101 için "prime number", 987972 için "not a prime number". 37 için yalnızca 3 ve 5, 101 için 3, 5, 7 ve 9 denenir.

Hoca bu aşamayı şöyle özetledi: İlk çözümde sayının yarısına kadar gidiliyordu. Sonra karekköküne kadar gidildi. Sonra 2'nin katları kontrol dışı bırakıldı. Böylece bir sayının asal olup

olmadığı çok daha az kontrolle bulunabildi. Hoca “daha iyisi var mı?” diye sordu ve “evet, var” diyerek bir sonraki yönteme geçti.

## 4. Eratosthenes Kalburu (Sieve of Eratosthenes)

### 4.1 Amaç ve Bedeli

Hocanın gösterdiği yöntem literatürde Eratosthenes Kalburu (Sieve of Eratosthenes) adıyla bilinir. Bu yaklaşım, verilen $n$ sayısının asal olup olmadığını bulurken aynı zamanda $n$'e kadar olan bütün asal sayıları da bulur.

Hoca, yöntemin bir dezavantajı olduğunu söyleyip bunun ne olduğunu sordu. Bir öğrencinin chat'ten verdiği cevap doğruydu: Bu yöntem ek bellek kullanır. Önceki çözümler sabit sayıda değişkenle çalışırken, bu yöntem $n$ boyutunda bir dizi kullanır. Hoca, işlem sayısını azaltmak için bellekten feragat edildiğini vurguladı.

### 4.2 Algoritmanın Mantığı

$n$'e kadar olan sayılar için bellekte bir dizi ayrılır ve dizinin bütün gözleri 0 yapılır. 0, o sayının olası bir asal sayı adayı olduğu anlamına gelir. Asal olma potansiyelini kaybeden sayılar 1 ile işaretlenir.

Adımlar:

- **2'nin katları:** 2 özel bir sayıdır, ayrı tutulur. 4'ten başlayarak ikişer ikişer ilerlenir ve 4, 6, 8, 10, ... 1 ile işaretlenir. Bunların asal olma ihtimali kalmamıştır.
- **Sınır:** $n$'in karekökü alınır ve `nn` değişkenine atanır. $n = 36$ ise `nn` $= 6$'dır.
- **Dış döngü:** $i = 3$'ten başlar, ikişer artar ve `nn`'ye kadar gider. 3'ten başlanır, çünkü 2'nin katları zaten işaretlenmiştir.
- **Asal sınaması:** `a[i]` hâlâ 0 ise $i$ asaldır.
- **İç döngü:** $i$ asalsa, $i \cdot i$'den başlanır ve $2i$ adımla $n$'e kadar ilerlenerek bütün uğranan sayılar 1 yapılır.
- **Sonuç:** Sonunda değeri 0 kalan sayılar asaldır. Bunlar yazdırılınca $n$ dahil $n$'e kadar bütün asal sayılar elde edilir.

### 4.3 Neden $i \cdot i$'den Başlanır, Neden $2i$ Adım Atılır?

Hoca bu iki seçimi örnekle açıkladı.

**$i = 3$:** İşaretlemeye $3 \cdot 3 = 9$'dan başlanır. Neden? 4, 6 ve 8 zaten 2 ile işaretlenmiştir. Aradaki 5 ve 7 zaten asaldır. 9'dan itibaren $2 \cdot 3 = 6$ adım atılır: $9 + 6 = 15$, sonra 21, 27, ... Bunların hepsi 1 yapılır. 3'er adım atılsaydı $9 + 3 = 12$'ye uğranırdı, oysa 12 zaten 2 ile işaretlenmiştir. Aynı işi tekrar yapmamak için 6'şar adım atılır.

**$i = 5$:** `a[5]` 0'dır, 5 asaldır. $5 \cdot 5 = 25$'ten başlanır ve $2 \cdot 5 = 10$'ar adımla ilerlenir: 25, 35, 45, 55, ... 5'er adım atılsaydı 30'a uğranırdı. 30 ise 2'nin katları gezilirken zaten işaretlenmiştir.

**$i = 7$:** `a[7]` 0'dır, 7 asaldır. $7 \cdot 7 = 49$'dan başlanır, çünkü 49'dan küçük katlar zaten işaretlenmiştir. $2 \cdot 7 = 14$ adımla doğrudan 63'e geçilir. 7 eklenseydi 56'ya uğranırdı, 56 da daha önce işaretlenmiştir.

| $i$ | Başlangıç ($i \cdot i$) | Adım ($2i$) | İşaretlenenler |
|---|---|---|---|
| 3 | 9 | 6 | 9, 15, 21, 27, ... |
| 5 | 25 | 10 | 25, 35, 45, 55, ... |
| 7 | 49 | 14 | 49, 63, 77, ... |

`nn`'nin anlamı: Dış döngü $\sqrt{n}$'e kadar gider. $n = 36$ için `nn` $= 6$'dır, yalnız 3 ve 5 incelenir. $n = 101$ için `nn` $= 10$'dur, 3, 5, 7 ve 9 incelenir. $n = 1000$ olsaydı `nn`, 1000'in karekökü olurdu. İç döngü ise her incelenen asal için $n$'e kadar gider.

**Ders sonrası not:** Hoca $n = 101$ için "3, 5, 7 ve 9'un her birinin peşinden iç döngüde gidecektik" dedi. 9 için iç döngü çalışmaz: 9, $i = 3$ turunda 1 ile işaretlenmiştir, bu yüzden `a[9] == 0` sınaması yanlış çıkar. Yani 101 için işaretleme yalnız 3, 5 ve 7 ile yapılır.

**Ders sonrası not:** Hoca sonuç için "1'den $n$'e kadar 0 gördüğünüz sayıları yazdırın" dedi. 0 ve 1 dizide hiç işaretlenmez, ama asal değildir. Yazdırma 2'den başlamalıdır.

### 4.4 Derste Sorulan Sorular

**Soru:** 3 ile işaretlerken 15, 21 gibi sayılar da algoritmaya dahil oluyor. Onları zaten 1'e eşitlemiştik, orada nasıl oluyor? **Cevap:** 1 ile işaretlenen sayılar asal olmadığı için işaretlenir. 2'nin turunda yalnızca 4, 6, 8, 10, … işaretlenmişti. 3'e geçince ilk işaretleme 9'dan başlar. 9'un asal olma potansiyeli ortadan kalkar. Sonra $9 + 2 \cdot 3 = 15$'e gidilir, 15 de 1 ile işaretlenir. Sonra 21'e, 27'ye gidilir. Bunlar 3'ün katları oldukları için asal olma potansiyellerini 3'ün turunda kaybeder. 15 ve 21 tek sayı olduğu için 2'nin turunda işaretlenmemiştir.

**Soru:** `nn` olarak ifade edilen değer nedir? **Cevap:** Asal olup olmadığına bakılan $n$'in karekökü-dür. Dış döngü 3'ten `nn`'ye kadar gider. 36 için 6'ya kadar, yani 3 ve 5'e bakılır. 101 için `nn` 10'dur, 3, 5, 7 ve 9'a bakılır. Dış döngü hangi sayıların peşinden gidileceğini belirler. İç döngü ise her biri için $n$'e kadar gider.

**Soru:** Adımın $2i$ olmasının matematiksel bir nedeni var mı? **Cevap:** Var. $i$'şer adım atılsaydı her ikinci adımda 2'nin bir katına uğranırdı. Örneğin 25'ten 30'a gidilirdi. 30 ise 2'nin turunda zaten ziyaret edilmiştir. $2i$ adımla 2'nin katları atlanır (bypass edilir), 25'ten doğrudan 35'e gidilir.

### 4.5 Kod

Aşağıdaki program dersteki akışı $n = 36$ için uygular:

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(void) {
    int n = 36;
    int i, j, nn;
    int *a = calloc(n + 1, sizeof(int)); /* 0: asal adayı, 1: asal değil */
    if (a == NULL)
        return 1;

    for (j = 4; j <= n; j += 2)          /* 2'nin katları */
        a[j] = 1;

    nn = (int)sqrt(n);
    for (i = 3; i <= nn; i += 2)
        if (a[i] == 0)
            for (j = i * i; j <= n; j += 2 * i)
                a[j] = 1;

    for (i = 2; i <= n; i++)             /* 0 ve 1 asal değil */
        if (a[i] == 0)
            printf("%d ", i);
    printf("\n");
    free(a);
    return 0;
}
```

Program `gcc -Wall kalbur.c -lm` ile uyarısız derlenir ve `2 3 5 7 11 13 17 19 23 29 31` yazdırır. Aynı yöntem 2 ile 1999 arasındaki her $n$ için deneme bölmesiyle karşılaştırılmış, sonuçlar aynı çıkmıştır.

## 5. Problem Çözerken İzlenecek Yol: Önce Doğru, Sonra Hızlı, Sonra Az Bellekli

Hoca asal sayı örneğinin genel bir ders verdiğini söyledi. Başta neredeyse bütün sayılar tek tek kontrol ediliyordu. Sonunda çok az kontrolle $n$'e kadar bütün asal sayılar bulunabiliyor. Bu, bir probleme bulunan ilk çözümün çok daha geliştirilebileceğini gösteren harika bir örnektir.

Bir problemle karşılaşınca izlenecek sıra:

- Önce brute force yaklaşımıyla ilk akla gelen çözümü üretin ve doğru çalışan bir çözüm elde edin.
- Sonra zaman karmaşıklığını, yani işlem sayısını azaltıp azaltamayacağınıza bakın.
- Sonra kullandığınız bellek miktarını azaltıp azaltamayacağınıza bakın.

  **Hocanın vurgusu:** İşlem sayısını azaltırken çoğu zaman bellekten belli oranda feragat etmek gerekir. Ortam buna müsaitse neden olmasın? Ama gömülü sistemlerde çalışıyorsanız, bellek sıkıntınız varsa ya da çok büyük bir veriyle çalışıyorsanız, ya veriyi bütün olarak değil parça parça işlemeniz ya da zaman karmaşıklığı daha yüksek bir çözümü seçmeniz gerekebilir.

Hoca buradan ileride görülecek bağlı listelere (linked list) bağlantı kurdu. Bir veri grubunu bellekte arka arkaya saklayamadığınızda, bellekte parça parça boş alanlar olduğunda hangi çözümlerin ve veri yapılarının kullanılabileceğini bağlı listeler gösterecektir. Bu konu önümüzdeki haftalarda işlenecek.

## 6. Arama Algoritmaları: Doğrusal Arama, İkili Arama ve Hash

### 6.1 Doğrusal Arama ve İkili Arama

Birinci sınıftan bilinen ilk yöntem doğrusal aramadır (linear search, diğer adıyla sequential search). Elemanlar tek tek kontrol edilir.

İkincisi daha sofistike olan ikili aramadır (binary search). İkili arama, böl ve yönet (divide and conquer) konusu altında anılan azalt ve yönet (decrease and conquer) yaklaşımının örneğidir. Yalnızca sıralı bir diziye uygulanabilir, ama adım sayısını çok değiştirir.

$n$ elemanlı bir dizide doğrusal arama en kötü durumda $n$ karşılaştırma yapar. İkili arama ise $\log_2 n$ mertebesinde karşılaştırma yapar. $n = 1024$ için doğrusal arama 1024 karşılaştırma yaparken ikili arama $\log_2 1024 = 10$ karşılaştırmayla aranan sayının dizide olup olmadığını ve yerini bulur.

Hoca buradan algoritmalardan ne beklendiğini açıkladı. Geliştirilen algoritmaların mümkünse doğrusal ($n$ mertebesinde) büyümesi, en azından $n^2$'den küçük olması tercih edilir. Daha da şanslıysanız $\log n$ mertebesinde büyümesi en çok istenendir. Doğrusal büyümede bir milyon eleman için bir milyon karşılaştırma yapılır. Oysa $\log_2 10^6 \approx 19{,}93$'tür (ara adım eklendi). Olay bir anda onlar, yirmiler, otuzlar mertebesine iner. Bir milyon yerine yaklaşık 20 adımda işi çözmek çok daha anlamlıdır.

### 6.2 Hash ile Arama

Arama algoritmalarının son noktası hash yaklaşımıdır. Aranan değer üzerinde belirli çarpma, bölme gibi matematiksel işlemler yapılarak bir sayı üretilir. Bu sayının gösterdiği gözdeki değerin aranan değer olup olmadığına bakılır. Böylece aranan eleman ortalama durumda tek adımda bulunur. $\log n$ çok avantajlı görünse de hash bundan da iyidir.

Hoca, hashing'in bu ders kapsamında görülmeyeceğini, Algoritma Analizi dersinde görüleceğini söyledi. Yine de hashing'e kısaca bakmanın bir şey kaybettirmeyeceğini, aramayı neden bu kadar etkili yaptığının kolayca anlaşılacağını ekledi.

Burada ilk oturum bitti ve ara verildi.

## 7. İkili Aramanın Akış Diyagramı ve Adım Adım İzlenmesi

### 7.1 Mantık

İkinci oturum ikili aramayla devam etti. Hoca, algoritmanın birinci sınıftan bilindiğini söyleyip kısa geçti.

- Aranan elemanın değeri dışarıdan alınır.
- Aranan grubun ortasındaki elemana bakılır. Ortadaki eleman aranan elemana eşit mi diye kontrol edilir.
- Eşit değilse aranan eleman ondan küçük mü büyük mü diye bakılır. Küçükse sola, büyükse sağa gidilir.
- Sola ya da sağa gidildiğinde elemanların yarısı elenmiş olur. Dizi sıralı olduğu için, ortadakinden küçük bir eleman sağ tarafta olamaz.
- Aranan eleman bulunduğunda ya da aranan değere uygun bir eleman olmadığından emin olunduğunda algoritma biter.

Her turda olası seçeneklerin yarısı elendiği için bu yaklaşıma azalt ve yönet denir.

### 7.2 Akış Diyagramı

- Başlangıç indisi `first = 0`, bitiş indisi `last = n - 1`'dir. Dizi 0'dan başladığı için son eleman $(n-1)$'inci gözdedir.
- `found` bir bayrak (flag) değişkenidir. Elemanın bulunup bulunmadığını saklar. Bu tür değişkenler genelde 0 ve 1 değerlerini alır. Başta 0 yapılır.
- `first <= last` olduğu ve `found` 0 kaldığı sürece döngü devam eder. Soldan gelen indis ile sağdan gelen indis adım adım birbirine yaklaşır.
- Döngüde `mid = (first + last) / 2` hesaplanır. Tam sayı bölmesi yapılır.
- `a[mid] == x` ise `found = 1` yapılır.
- Değilse ve `x < a[mid]` ise aranacak bölge sol yarıdır: `last = mid - 1`. Başlangıç değişmez.
- Değilse aranacak bölge sağ yarıdır: `first = mid + 1`. Son değişmez.
- Döngü bitince `found` 1 ise eleman yazdırılır, 0 ise "not found" yazdırılır.

Aşağıdaki fonksiyon bu akışı uygular. Bulursa indisi, bulamazsa $-1$ döndürür:

```c
/* a küçükten büyüğe sıralı olmalı */
int ikili_arama(const int a[], int n, int x) {
    int first = 0, last = n - 1, mid, found = 0;
    while (first <= last && found == 0) {
        mid = (first + last) / 2;
        if (a[mid] == x)
            found = 1;
        else if (x < a[mid])
            last = mid - 1;
        else
            first = mid + 1;
    }
    return found ? mid : -1;
}
```

Fonksiyon bir test programıyla `gcc -Wall` ile uyarısız derlendi. Sıralı bir test dizisinde 0-15 arası her değer için sonucu doğrusal aramayla karşılaştırıldı, sonuçlar aynı çıktı.

### 7.3 Derste İzlenen Örnek: 4'ün Aranması

Derste 8 elemanlı sıralı bir dizide 4 arandı. 4 dizide yoktur. Altyazıdan dizinin şu elemanları çıkıyor: `a[1] = 3`, `a[2] = 5`, `a[3] = 6`.

| Adım | first | last | mid | Karşılaştırma | Sonuç |
|---|---|---|---|---|---|
| 1 | 0 | 7 | 3 | `a[3] = 6`, $4 \neq 6$, $4 < 6$ | `last = 2` |
| 2 | 0 | 2 | 1 | `a[1] = 3`, $4 \neq 3$, $4 > 3$ | `first = 2` |
| 3 | 2 | 2 | 2 | `a[2] = 5`, $4 \neq 5$, $4 < 5$ | `last = 1` |

| Adım | first | last | mid | Karşılaştırma | Sonuç |
|---|---|---|---|---|---|
| 4 | 2 | 1 | - | `first <= last` yanlış | döngüden çıkılır |

Adım 1'de `mid` $= (0+7)/2 = 3{,}5$, tam sayı bölmesiyle 3'tür. Adım 2'de $(0+2)/2 = 1$, adım 3'te $(2+2)/2 = 2$'dir. Döngüden çıkıldığında `found` hâlâ 0'dır, bu yüzden "not found" yazdırılır.

Hoca ilk adımdan sonra ters durumu da anlattı: 12 aranıyor olsaydı 12, `a[mid]`'den küçük olmadığı için sağa gidilirdi. `first`, `mid + 1` yani 4 olurdu, `last` değişmezdi. Arama 4 ile 7 arasında sürerdi.

> **Ders sonrası not:** Hoca bu örnekte "12, 8'den küçük olmadığı için" ve "mid değerine 1 koyacaktık" dedi. O anda ortadaki eleman 6'dır (`a[3] = 6`). Doğrusu: 12, 6'dan küçük olmadığı için `first = mid + 1 = 4` yapılır. Hoca adım 2'den sonra da bir an `first` ile `mid`'i karıştırıp düzeltti. Tablo düzeltilmiş hali gösterir.

Hoca buradan genel bir gözlem çıkardı: Her turda ya başlangıç indisi artar ya da son indis azalır. İkisi birbirini geçtiği anda, yani `first > last` olduğunda döngü biter.

Hoca bir de alternatif gösterdi: `found` bayrağını içeride kontrol etmek yerine, "ortadaki eleman aranana eşit olmadığı sürece" koşulu doğrudan döngü koşuluna yazılabilir. Bu da olası çözümlerden biridir.

Hoca ikili arama için soru sordu, soru gelmedi.

## 8. Algoritma Analizi: Temel İşlemler ve Big O

### 8.1 Buraya Kadar Ne Görüldü?

Hoca, şimdiye kadar temel sıralama algoritmalarının, arama algoritmalarının ve akış diyagramı sembollerinin üzerinden geçildiğini hatırlattı. Bir algoritmanın verimliliğinin nasıl artırılabileceği asal sayı örneğiyle gösterilmişti. Sıra, ilk derste söylenen iki başlık altında bir algoritmanın nasıl analiz edileceğine geldi: zaman karmaşıklığı (time complexity) ve yer karmaşıklığı (space complexity).

### 8.2 Temel İşlemler (Basic Operations)

Bir kod parçasına ya da akış diyagramına bakınca göze çarpan temel işlemler (basic operations, primitive operations) şunlardır:

- Atama işlemi.
- Aritmetik işlemler: toplama, çıkarma, çarpma, bölme.
- Karşılaştırma işlemleri: küçüktür, büyüktür, eşit midir, eşit değil midir.
- Değişken tanımlama.
- Belleğe erişim. Hoca, belleğe erişimin de diğer işlemler kadar önemli olduğunu söyledi.

Bu işlemlerin işlemcide kaç döngüde (cycle) gerçekleştiği birbirinden farklıdır. Ama analizde bununla ilgilenilmez. Her biri bir birim iş sayılır. Önemli olan kaç kez gerçekleştikleridir.

### 8.3 Bir Döngü Örneğinde İşlem Sayımı

Hoca slaytın sağ üst köşesindeki bir kod parçasının işlemlerini saydı:

- 3 değişken tanımlama,
- $n + 2$ atama,
- $n$ karşılaştırma,
- $n$ artırma,
- $n$ çarpma.

Toplam: $3 + (n + 2) + n + n + n = 4n + 5$ işlem.

### 8.4 Big O ve Sadeleştirme

Big O gösterimi, bir akışta ya da sözde kodda gerçekleşen temel işlemlerin sayısını bulup bu sayıyı sadeleştirerek elde edilen ifadedir. Sadeleştirme iki kuraldan oluşur:

- Sabit sayılar atılır.
- Küçük dereceli terimler atılır.

$4n + 5$'te 5 hem sabit hem küçük dereceli olduğu için atılır. $n$'in başındaki 4 de sabit olduğu için atılır. Sonuç $O(n)$'dir.

Neden atılır? $n$ genelde girdi sayısını ifade eder. $n$ büyüdükçe, sonsuza götürüldüğünde, 4'ün ya da 1000, 10.000 gibi herhangi bir sabitin önemi kalmaz.

Başka bir örnek: $n^2 + 100n$. 100 sabittir, $n$ de $n^2$'den çok küçüktür. Sayılar büyüdükçe $100n$ terimi önemsizleşir ve ifade $O(n^2)$ olarak yazılır.

Hoca, bunun ayrıntılarının gelecek sene Algoritma Analizi dersinde görüleceğini, ama öğrencilerin kendi tasarladıkları algoritmaların karmaşıklığı hakkında bir şey söyleyebilmeleri için temel prensiplerin bu dersin başında verildiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Big O'nun biçimsel tanımı şöyledir. $f(n) = O(g(n))$ demek, öyle pozitif bir $c$ sabiti ve bir $n_0$ değeri vardır ki her $n \geq n_0$ için $f(n) \leq c \cdot g(n)$ olur demektir. Örneğin $n \geq 5$ için $4n+5 \leq 5n$ olduğundan $4n+5 = O(n)$'dir ($c = 5$, $n_0 = 5$). Sabitlerin atılabilmesi bu tanımdaki $c$'den gelir.

## 9. En İyi, Ortalama ve En Kötü Durum

Bir algoritma zaman ve yer karmaşıklığı açısından üç durum için incelenebilir: en iyi durum (best case), ortalama durum (average case) ve en kötü durum (worst case). Hoca en iyi ve en kötü durumu örneklerle anlattı.

### 9.1 Doğrusal Arama

**Sırasız dizi:** En iyi durumda aranan eleman ilk adımda bulunur. Örneğin 1 aranıyor ve ilk eleman 1'dir. Bu durum $n$'e bağlı değildir. Bir milyon eleman olsa da en iyi durumda ilk adımda bulma potansiyeli vardır. Bu yüzden en iyi durum karmaşıklığı $O(1)$'dir. En kötü durumda ise eleman ya en sondadır ya da dizide hiç yoktur. Bir milyon eleman için bir milyon kontrol gerekir. En kötü durum $O(n)$'dir.

> **Ders sonrası not:** Hoca toparlarken bir yerde "linear search'ün best case için karmaşıklığı $O(1)$ iken worst case için karmaşıklığı $O(1)$'dir" dedi. Bu bir dil sürçmesidir. Hemen öncesinde doğru söylendiği gibi en kötü durum $O(n)$'dir.

**Sıralı dizi:** Dizi sıralı da olsa en iyi durumda eleman ilk adımda bulunursa karmaşıklık $O(1)$'dir. En kötü durumda yine aranan eleman en sonda olabilir ya da hiç olmayabilir. Örneğin 8 aranıyorsa 1, 2, 3, 4, 5 kontrollerini tek tek yapmak gerekir. Sıralı dizide doğrusal aramanın en kötü durum karmaşıklığı da $O(n)$'dir.

### 9.2 En Küçük ve En Büyük Elemanı Bulma

Hoca bu örneğin, elemanların sıralı olup olmamasının karmaşıklığı nasıl etkilediğini gösterdiğini söyledi.

**Sırasız dizi:** En küçük ve en büyük elemanı bulmak için hem en iyi hem en kötü durumda $n$ karşılaştırma yapmak gerekir. En iyi durum için $O(1)$ denemez, çünkü ilk bakılan elemanın en küçük ya da en büyük olduğu bilinemez. Bütün elemanlara bakmak zorunludur. Doğrusal aramadan ayrıldığı nokta budur. Hem en iyi hem en kötü durum $O(n)$'dir.

**Sıralı dizi:** Dizi küçükten büyüğe sıralıysa ilk eleman en küçük, son eleman en büyüktür. Bunun için dizinin hangi yönde sıralı olduğunu bilmek önemlidir. Hiçbir kontrol, hiçbir döngü gerekmez. Yapılan tek şey ilk elemanı en küçük, son elemanı en büyük olarak yazdırmaktır. Hem en iyi hem en kötü durum $O(1)$'dir. Bunun için dizinin sıralı olduğunun bilinmesi şarttır.

| Problem | Girdi | En iyi durum | En kötü durum |
|---|---|---|---|
| Doğrusal arama | Sırasız | $O(1)$ | $O(n)$ |
| Doğrusal arama | Sıralı | $O(1)$ | $O(n)$ |
| En küçük ve en büyük | Sırasız | $O(n)$ | $O(n)$ |
| En küçük ve en büyük | Sıralı | $O(1)$ | $O(1)$ |

Hoca, ortalama durumun bu derste hiç konuşulmayacağını, Algoritma Analizi dersinde ele alınacağını söyledi.

## 10. Zaman Karmaşıklığını Belirleyen Etkenler

Bir algoritmanın analizi yapılırken zaman ve yer karmaşıklığı girdi cinsinden üç temel notasyonla ifade edilir. Bunların en önemlisi Big O'dur. Big Omega ve Big Theta da vardır, ama literatürde çoğunlukla Big O kullanılır.

Analizde temel işlemler sayılır ve karmaşıklığa buna göre karar verilir. Ama asıl belirleyici olan ve işi kolaylaştıran döngülerdir. $n$ cinsinden ifade edilen döngülerde tek döngü $n$, iç içe iki döngü $n^2$, iç içe üç döngü $n^3$ verir. İşlemler sözde kodda, bir programlama dilinde yazılmış kodda ya da bir akış diyagramında sayılabilir. Hangisinde sayıldığının hiçbir önemi yoktur.

Zaman karmaşıklığını etkileyen iki temel bilgi vardır:

- **Girdi sayısı:** Genelde $n$ ile ifade edilir. Girdiler $k$, $m$, $d$ gibi başka adlarla verildiyse karmaşıklık da onlarla yazılır: $O(k)$, $O(m)$, $O(n \cdot m)$ gibi. Hangi girdilerin belirleyiciliği varsa onlar kullanılır. Ad verilmediyse genelde $n$ üzerinden formüle edilir.
- **Girdinin düzeni ve tipi:** Az önceki örneklerde görüldüğü gibi girdinin sıralı ya da sırasız olması karmaşıklığı doğrudan etkileyebilir.

### 10.1 Ardışık ve İç İçe Döngüler

Hoca slayttaki örneklerle şu kuralları gösterdi:

- İç içe iki döngü $O(n^2)$ verir.
- Döngüler iç içe değil de arka arkaya olsaydı, örneğin 3, 4, 5 tane, $O(3n)$, $O(5n)$ gibi ifadeler çıkardı. 10 tane ardışık tek döngü olsa bile $O(10n)$'deki 10 sabit olduğu için düşer ve sonuç $O(n)$ olur. Asıl belirleyici olan döngülerin iç içe olmasıdır.
- İç içe üç döngü $O(n^3)$ verir.
- $n^2 + 5n + 3$ gibi bir işlem sayısı hesaplandıysa küçük dereceli terimler ve sabitler atılır ve $O(n^2)$ yazılır.
- Aynı şekilde $n^3$'lü bir ifadede $n^2$, $n$ ve sabitler önemini kaybeder, ifade $O(n^3)$ olarak yazılır.

  **Hocanın vurgusu:** Bir O notasyonunda, özel anlatım durumları dışında, hiçbir zaman sabit bir sayı ya da küçük dereceli bir terim görmezsiniz.

## 11. Büyüme Dereceleri

Hoca asimptotik notasyona geçmeden önce büyüme derecelerini (orders of growth) sıraladı:

- $O(1)$**:** En sevilen büyüme derecesidir, çünkü iş tek adımda çözülür.
- $O(\log n)$**:** Oldukça yavaş büyür. İkili arama buna örnektir.
- $O(n)$**:** Yine değerli bir büyüme derecesidir. Doğrusal zamanda çözüm varsa çok iyi bir çözüm demektir.
- $O(n \log n)$**:** Heap sort, merge sort, quick sort gibi birçok sıralama algoritması bu mertebededir. $n^2$'ye göre tercih edilir.
- $O(n^2)$**:** $n$ çok büyüdüğünde, örneğin bir milyon olduğunda, bir milyon çarpı bir milyon, yani $10^{12}$ iş demektir. Polinom zamanda çözüm sıkıntıya girmeye başlar.
- $O(n^3)$**:** Aynı sorun daha da büyüktür.

- $O(2^n)$ **ve** $O(n!)$**:** Hocaya göre bunları konuşmaya bile gerek yoktur. $n$ 100 seçildiği anda en güçlü bilgisayarlar bile bu çözümleri makul sürede bitiremez.

  **Ders sonrası not:** Hoca quick sort'u heap sort ve merge sort ile birlikte $n \log n$ grubunda saydı. Quick sort ortalama durumda $O(n \log n)$'dir, en kötü durumda $O(n^2)$'dir. Heap sort ve merge sort her durumda $O(n \log n)$'dir.

Hocanın hedefi: $n^3$'ün, mümkünse $n^2$'nin altında büyüyen çözümler üretmek ve karşılaştırma yaparken bu çözümlerin diğerlerine göre avantajını görebilmek.

### 11.1 Örnek: Toplamı 12 Olan Sayıları Bulmak

Slayttaki örnekte 1, 3, 2, 8, 6, 4 sayıları veriliyor. Problem şu: Bu sayılardan hangileri kullanılırsa toplam 12 elde edilir? Hoca iki olası cevap söyledi: 8 ile 4 olabilir, 6, 3, 1, 2 de olabilir.

Bunu bulmak için bu kümeye dahil sayıların bütün alt kümelerine bakmak gerekir: teklilere, ikililere, üçlülere, dörtlülere. Kaç eleman varsa $2^n$ işlem demektir. Herkesin aklına gelebilecek bu basit çözümle hepsi tek tek kontrol edilirse, bu küçücük örnekte bile $2^6 = 64$ alt küme, boş küme çıkarılınca 63 karşılaştırma gerekir. $n$ 100 olduğunda problem büyük sayılar için hiçbir şekilde çözülemez.

Hoca, kombinatorik problemlerde özel bir çözüm yoksa ortaya çıkan çözümlerin polinom zamandan çok uzak olduğunu ya da yalnızca çok küçük örnek setlerinde çalışabildiğini söyledi. Bunlar için yeni bakış açıları kullanılır. Bunların bir kısmı bu derste, bir kısmı gelecek seneki derste görülecek.

Hoca büyüme dereceleri için soru sordu, soru gelmedi.

## 12. Asimptotik Notasyon: Big O, Big Omega, Big Theta

Asimptotik notasyonun (asymptotic notation) altında üç önemli notasyon vardır:

- **Big O:** Üst sınırı (upper bound) belirler. $O(n^3)$ dendiğinde fonksiyonun $n^3$'ü aşmayacağı bir sınır verilmiş olur. Üst sınır daha önemli olduğu için en çok bu notasyon tercih edilir.
- **Big Omega:** Fonksiyonun alt sınırını (lower bound) ifade eder. Çok sık kullanılan bir notasyon değildir.
- **Big Theta:** Sıkı sınırı (tight bound) ifade eder. Slayttaki çizimde üç fonksiyon vardı: üst sınır $F_1$, alt sınır $F_3$. Arada kalan $F_2$ fonksiyonu, üç fonksiyon aynı büyüme derecesine sahipse Big Theta ile ifade edilir.

Üst sınır verilirken önemli olan, sınırı en yakın ve en makul değerle ifade etmektir. $n^3$'lük bir algoritma için $n^4$ ya da $n^5$ de bir üst sınır olarak tanımlanabilir, ama bu hiçbir zaman tercih edilmez.

> **Hocanın vurgusu:** Omega ve Theta bu senenin konusu değildir. Sorumlu olunan kısım, Big O notasyonunu kullanarak bir akış diyagramının ya da sözde koddaki bir kod parçasının karmaşıklığını ifade edebilmektir.

## 13. Sözde Kod ve En Büyük Elemanı Bulma

### 13.1 Sözde Koddan Korkmayın

Hoca, öğrencilerin sözde koddan (pseudo code) genelde uzak durduğunu, ama aslında bir programlama dili kadar kolay yazıldığını söyledi. Sözde kod için farklı notasyonlar kullanılır:

- Atamada eşittir yerine ok işareti (←) kullanan notasyonlar vardır.
- `for` için "başlangıç `to` bitiş `do`" kalıbını kullanan notasyonlar vardır.
- Karşılaştırma ve aritmetik işlemler için standart işaretler kullanılır.

Sözde kod, daha sözel ifadelerin kullanıldığı, parantezlerden pek yararlanılmayan, iyice sadeleştirilmiş, konuşma diline yakın bir gösterimle akıldaki algoritmayı ifade etmektir.

### 13.2 En Büyük Elemanı Bulma

Slaytın solunda en büyük elemanı bulan bir sözde kod, sağında temel işlem sayımı vardı. Altyazıdaki anlatıma göre sözde kod şu yapıdadır:

```
max ← A[0]
for i ← 1 to n - 1 do
    if A[i] > max then
        max ← A[i]
return max
```

İşlem sayımı:

- `max ← A[0]`: dizinin gözüne bir erişim ve bir atama, yani 2 işlem.
- `for` döngüsü: $i$'ye başlangıç ataması ve $n-1$ sınırıyla ilgili atama yapılır. Döngü kontrolünün $n$ kez gerçekleştiği görülür.
- Döngü içinde `A[i] > max` karşılaştırması da döngü sayısı kadar gerçekleşir.

Bütün işlemler toplanıp sabitler atılınca en büyük elemanı bulma algoritmasının karmaşıklığı $O(n)$ çıkar. Tek bir `for` döngüsü olduğuna bakarak da bu rahatlıkla söylenebilir.

> **Hocanın vurgusu:** Bazı sözde kodlarda döngü değişkeni ikişer ikişer atlar ya da ikinin katları şeklinde, logaritmik ilerler. Oralarda dikkatli olmak, karmaşıklığı gerekirse logaritma cinsinden ifade etmek gerekir.

Bunun dışında genelde iç içe döngü sayısı, $n$'in üssünü belirler. İç içe iki döngü varsa genelde $n^2$'den söz edilir. Aynı kodda iç içe iki döngü ve ayrıca beş tane tek döngü varsa ifade $O(n^2 + 5n)$ gibi görünür, ama $5n$, $n^2$'den küçük olduğu için atılır ve sonuç yine $O(n^2)$'dir.

## 14. Teorik Analiz ve Deneysel Ölçüm

### 14.1 Teorik Analiz

Şimdiye kadar anlatılan yöntem teorik analizdir. Yazılımdan ve donanımdan tamamen bağımsızdır. İki algoritma yalnızca girdi sayısı üzerinden karşılaştırılır ve hangisinin daha iyi olduğu söylenebilir. Tercih edilme sebebi de bu bağımsızlıktır: Kod yazılmaz, kod çalıştırılıp süre ölçülmez. Süre ölçerken işletim sisteminin o anda başka bir yük yüzünden daha yavaş çalışıp çalışmadığı gibi dertlere girilmez. Bir problem için üretilen iki çözümü objektif olarak ve emek harcamadan karşılaştırmanın en iyi yolu, temel işlemleri sayarak teorik analiz yapmaktır.

### 14.2 Deneysel Ölçüm

Bazen elde hazır iki kod parçası vardır ve bunlar çok uzundur, örneğin 40-50 bin satır. Bu durumda her döngüyü tek tek incelemek yerine iki kod aynı donanım ve yazılım koşullarında çalıştırılır:

- Farklı girdi sayıları için çalıştırılır: $n = 10$, $100$, $1000$, $10.000$ gibi.
- Her girdi sayısı için 10 ya da 20 kez çalıştırılıp ortalama alınır.
- Ölçülen sürelerle girdi boyutu ile süre arasındaki ilişkiyi gösteren bir grafik çizilir.

Bu grafik iki kodun verimliliği hakkında çok şey söyler ve hangisinin seçileceğine buna göre karar verilir.

### 14.3 Küçük Girdilerde Sonuç Tersine Dönebilir

Analizde her zaman en kötü durum senaryosu ve büyük girdi sayıları, örneğin 100 milyon eleman, esas alınır. Ama bazı özel durumlarda A algoritması girdi sayısı çok büyükken B'den çok daha iyi sonuç verirken, girdi sayısı küçükken durum tersine dönebilir. Bunun sebebi Big O'da atılan sabitlerdir. Örneğin $100n + n^2$'deki 100 küçük girdilerde etkilidir.

> **Hocanın vurgusu:** Sizin durumunuzda hep küçük girdilerle çalışılıyorsa, teorik analiz tersini söylese de küçük girdilerde avantajlı olan algoritmayı seçmelisiniz. Seçim biraz da kullanım durumunuza bağlıdır.

## Örnek Sorular

Bu derse ait hazır kare ve video yoktur. Aşağıdaki sorular derste sözlü olarak sorulan ya da slayttan sözlü aktarılan sorulardır.

**Soru 1** (derste sözlü, 00:27:44): Eratosthenes Kalburu yaklaşımının dezavantajı nedir?

> **Çözüm:** Ek bellek kullanmasıdır. Deneme bölmesine dayalı yöntemler sabit sayıda değişkenle çalışır. Kalbur ise $n$'e kadar her sayı için bir göz tutan $n$ boyutlu bir dizi kullanır. İşlem sayısındaki kazanç, bu bellek bedeliyle elde edilir. Bu cevap derste bir öğrenci tarafından verildi, hoca onayladı.

**Soru 2** (derste sözlü, 00:39:04): Kalburda 3 ile işaretlerken 15, 21 gibi sayılara da uğranıyor. Bunlar zaten 1 yapılmamış mıydı?

> **Çözüm:** Hayır. 2'nin turunda yalnızca çift sayılar (4, 6, 8, 10, ...) işaretlenmiştir. 15 ve 21 tektir, bu yüzden ilk kez 3'ün turunda işaretlenir. 3'ün turu 9'dan başlar ve 6'şar adımla 9, 15, 21, 27, ... sayılarını 1 yapar. Bunlar 3'ün katları oldukları için asal olma potansiyellerini bu turda kaybeder.

**Soru 3** (derste sözlü, 00:39:59): Kalburdaki `nn` değeri nedir?

> **Çözüm:** `nn`, $n$'in kareköküdür ve dış döngünün sınırıdır. $n = 36$ için `nn` $= 6$, dış döngü 3 ve 5'i inceler. $n = 101$ için `nn` $= 10$, dış döngü 3, 5, 7 ve 9'u inceler. 9 asal olmadığı için (`a[9] = 1`) iç döngü 9 için çalışmaz. İç döngü her asal için $n$'e kadar gider.

**Soru 4** (derste sözlü, 00:41:48): Kalburda adımın $2i$ olmasının matematiksel bir nedeni var mı?

> **Çözüm:** Var. $i$'şer adım atılsaydı her ikinci adımda 2'nin bir katına uğranırdı. Bu sayılar 2'nin turunda zaten ziyaret edilmiştir. $2i$ adımla bunlar atlanır. Derste verilen örnek: $i = 5$ için 25'ten 30'a değil, doğrudan 35'e gidilir.

**Soru 5** (derste sözlü, slayttan aktarıldı, 01:25:09): 1, 3, 2, 8, 6, 4 sayılarından hangileri seçilirse toplam 12 elde edilir? Kaba kuvvetle kaç alt küme denenir?

Derste iki cevap söylendi: {8, 4} ve {6, 3, 1, 2}. Kaba kuvvet çözüm bütün alt kümeleri dener: $2^6 = 64$ alt küme, boş küme hariç 63.

> **Çözüm (derste verilmedi):** Bütün alt kümeler python ile tek tek denendi. Toplamı 12 olan tam 4 alt küme vardır: {8, 4}, {1, 3, 8}, {2, 6, 4} ve {1, 3, 2, 6}. Derste söylenen iki cevap bunların arasındadır.

## Dersin Toparlaması

Ders, geçen hafta yarım kalan insertion sort ile başladı. Insertion sort, swap yapmadan yalnızca gerektiği kadar kaydırma yaparak her elemanı solundaki sıralı kısma yerleştirir. Sıralı dizide $O(n)$, ters sıralı dizide $O(n^2)$ çalışır. Döngü koşulunda sınır kontrolünün ilk sırada yazılması gerektiği vurgulandı. Ardından asal sayı problemi adım adım iyileştirildi: $x-1$'e kadar bölmekten yarıya, kareköke ve yalnız tek bölenlere geçildi. Son olarak Eratosthenes Kalburu ile ek bellek karşılığında $n$'e kadar bütün asal sayılar bulundu. Bu örnekten şu ilke çıkarıldı: Önce doğru çalışan bir çözüm, sonra daha az işlem, sonra daha az bellek. Zaman kazancı çoğu zaman bellekle ödenir.

İkinci yarıda doğrusal arama ile ikili arama karşılaştırıldı ($n$ ile $\log_2 n$), hash'e kısaca değinildi ve ikili aramanın akışı bir örnekle izlendi. Sonra algoritma analizine giriş yapıldı: temel işlemler sayılır, sabitler ve küçük dereceli terimler atılarak Big O elde edilir. Karmaşıklığı asıl belirleyen döngülerdir, girdinin sıralı olup olmaması da sonucu değiştirir. Büyüme dereceleri $1$'den $n!$'e sıralandı, asimptotik notasyonlar tanıtıldı ve teorik analiz ile deneysel ölçüm karşılaştırıldı. Bu dönem sorumlu olunan kısım, bir akış diyagramının ya da sözde kodun karmaşıklığını Big O ile ifade edebilmektir. Hoca, sonraki haftalarda bağlı listelerin işleneceğini söyledi.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.